

FINAL WRITTEN QUIZ CASE — UNIVERSITY COURSE ENROLLMENT SYSTEM

The School of Computer Science operates a **Course Enrollment System** that ensures students enroll **only in courses they are eligible for**.

This system supports **three types of courses**:

- **TheoryCourse** – regular lecture-based courses
- **LabCourse** – courses that involve laboratory activities
- **ProjectCourse** – advanced project-based courses

Students must fulfill certain requirements before enrolling in specific types of courses.

Program Requirements (Console Application)

1. Initial Start

When the program starts, the program will do the following tasks:

A. System must **pre-load** the following **courses**:

Type	Code	Name	Capacity	Credit
TheoryCourse	COMP6820	Object Oriented Programming	40	5
TheoryCourse	COMP7011	Artificial Intelligence	30	8
LabCourse	CLAB6820	Object Oriented Programming	40	5
LabCourse	CLAB7010	Artificial Intelligence	30	5
ProjectCourse	CPRJ8888	Web Programming	35	10
ProjectCourse	CPRJ9999	Mobile Programming	35	10

B. Prompt the user to input their **student credentials to log-in**. Here is all the data the student must provide.

- **Student ID** – Validate the Student ID must be 10 characters long.
- **Name** – Validate the Name must be at least 3 characters long.
- **Age** – Validate the Age must be at least 17 years old.

```
Please input your student data!
Student ID (must be 10 characters long): 250988441
Student ID (must be 10 characters long): 2509884411
Name (min 3 characters): El
Name (min 3 characters): Eleanor
Age (at least 17): 16
Age (at least 17): 17

Select Your Action
1. Enroll Course
2. Attend Safety Training
3. View Enrolled Course
4. Exit
>> |
```

Figure 1. Student Credentials

2. Student Actions

After the student has logged in, show the following menu:

A. Enroll Course

- **Display all** the available **courses** in the application **indexed from 1 to N** (numbers of available courses).
- Prompt the user to **choose** what course they want to enroll in. Validate the input must be **between 1 to N or 0 to exit** the process.
- If user chooses **0**, **stop the process** and go back to student menu.
- If user chooses **a course**, validate if the user is **eligible to enroll** in that course based on the following criteria:
 - The Course's **capacity** is **available**.
 - The Course has **not been enrolled** by this student.
 - **Theory Course**
All students are **eligible** to enroll this course.
 - **Lab Course**
Only students who **have attended the safety training** are eligible to enroll on this course.
 - **Project Course**
Only students whose **credits are 10 or above** are eligible to enroll in this course

```

no. code - name - capacity - credit
=====
1. COMP6820 - Object Oriented Programming - 40 - 5
2. COMP7011 - Artificial Intelligence - 30 - 8
3. CLAB6820 - Object Oriented Programming - 40 - 5
4. CLAB7010 - Artificial Intelligence - 30 - 5
5. CPRJ8888 - Web Programming - 35 - 10
6. CPRJ9999 - Mobile Programming - 35 - 10
=====
Choose course to enroll (1 - 6)[0 to exit]: 1
Eleanor enrolled in COMP6820-Object Oriented Programming → SUCCESS
Press any key to continue..

```

Figure 2. Enroll Success

```

no. code - name - capacity - credit
=====
1. COMP6820 - Object Oriented Programming - 40 - 5
2. COMP7011 - Artificial Intelligence - 30 - 8
3. CLAB6820 - Object Oriented Programming - 40 - 5
4. CLAB7010 - Artificial Intelligence - 30 - 5
5. CPRJ8888 - Web Programming - 35 - 10
6. CPRJ9999 - Mobile Programming - 35 - 10
=====
Choose course to enroll (1 - 6)[0 to exit]: 3
Eleanor enrolled in CLAB6820-Object Oriented Programming → (FAILED – requires safety training)
Press any key to continue..

```

Figure 3. Enroll Failed (Lab Course)

```

no. code - name - capacity - credit
=====
1. COMP6820 - Object Oriented Programming - 40 - 5
2. COMP7011 - Artificial Intelligence - 30 - 8
3. CLAB6820 - Object Oriented Programming - 40 - 5
4. CLAB7010 - Artificial Intelligence - 30 - 5
5. CPRJ8888 - Web Programming - 35 - 10
6. CPRJ9999 - Mobile Programming - 35 - 10
=====
Choose course to enroll (1 - 6)[0 to exit]: 5
Eleanor enrolled in CPRJ8888-Web Programming → (FAILED – credits below minimum requirement)
Press any key to continue..

```

Figure 4. Enroll Failed (Project Course)

B. Attend Safety Training

- If the student **has not attended** safety training before, update their safety training status to **have attend the training** and show a successful attendance message.
- If the student has **already attended** a safety training before, show a message that they **have already attended** a safety training and go back to student menu.

```
Select Your Action
1. Enroll Course
2. Attend Safety Training
3. View Enrolled Course
4. Exit
>> 2
Eleanor has successfully attempted safety training
```

Figure 5. Safety Training (Before)

```
Select Your Action
1. Enroll Course
2. Attend Safety Training
3. View Enrolled Course
4. Exit
>> 2
Eleanor has already attempted safety training!
```

Figure 6. Safety Training (After)

C. View Enrolled Course

- Display all the **course information** that the student has enrolled.
- Display the **total credit** that the student has from all the **courses they enrolled in**.

```
no. code - name - capacity - credit
=====
1. COMP6820 - Object Oriented Programming - 40 - 5
2. CLAB7010 - Artificial Intelligence - 30 - 5
=====
Total credit: 10
Press any key to continue..
```

Figure 7. View Enrolled Course

D. Exit

- **Exit** the program.

System Requirements (to be implemented in your code)

1. Course Abstraction

The system must define an **abstract class Course** containing:

- code (String)
- name (String)
- capacity (int)
- credit (int)
- an abstract method:
enroll(boolean)

Three course types must extend Course and implement enroll (Student):

a. TheoryCourse

Always allows enrollment (always returns true).

b. LabCourse

Requires the student to have completed **safety training**

c. ProjectCourse

Requires the student to have **at least 10 accumulated credits** from previously enrolled courses

2. Student Class

A Student must contain:

- studentId (String)
- name (String)
- age (int)
- hasSafetyTraining (boolean)
- enrolledCourses (ArrayList/Vector)
- creditTaken (int)

The student must provide these behaviors:

1. attemptEnroll(Course c)
2. attemptSafetyTraining()
3. getCredits()

3. EnrollmentService

A service class responsible for **validating course enrollment**.

4. SafetyTrainingService

A service that handles **safety training attempts**.

What Students Must Submit

Students must produce:

1. A working Java console application
2. Implementation of:
 - Abstract classes
 - Inheritance
 - Polymorphism
 - Encapsulation
3. Use of:
 - EnrollmentService
 - SafetyTrainingService
4. Correct logic for:
 - Enrollment validation per course type
 - Safety training requirement
5. Implementation of SOLID principles
6. Menu-based interaction